

Learning Module: Tableau Desktop using `WAREHOUSE_SAMPLE`

0. First... what you're really working with

Your database is not just OLTP. It's layered:

- **OLTP layer (dbo)** → raw transactions
`Orders`, `OrderDetails`, `Customers`, `Products`, etc.
- **Analytical views**
 - `vw_FactSales`
 - `vw_SalesAnalysis`
- **Data Warehouse (DW schema)** → star schema
 - `FactSales`
 - `DimCustomer`, `DimProduct`, `DimEmployee`
 - `AggSalesMonthly`
- **ETL pipeline**
 - `STG` tables
 - stored procedures (`sp_Load_DW`)

So... you're basically holding a mini enterprise BI system.

That matters because Tableau learning becomes *much more meaningful* when you use the **right layer**.

1. Module 1 — Connecting Tableau to SQL Server

Goal

Get data into Tableau correctly.

Task

- Open Tableau Desktop
- Connect → Microsoft SQL Server
- Select database: `WAREHOUSE_SAMPLE`

What to use (important decision)

Start with:

- `DW.FactSales` (main fact table)
- Join with:
 - `DW.DimCustomer`
 - `DW.DimProduct`
 - `DW.DimEmployee`

This gives you a **clean star schema**, which Tableau works best with.

Avoid starting with raw tables like `Orders` unless you're practicing joins.

2. Module 2 — Understanding Measures vs Dimensions

Goal

Stop dragging fields blindly.

From your schema:

Dimensions (categorical)

- CustomerName
- City, Country
- ProductName
- CategoryID
- EmployeeName
- OrderDate (as discrete)

Measures (numeric)

- SalesAmount
- Quantity
- Price

Exercise

Build:

- Total Sales → SUM(SalesAmount)
- Total Quantity → SUM(Quantity)
- Sales by Product
- Sales by Customer

Just drag and observe how Tableau aggregates automatically.

3. Module 3 — Time-Based Analysis

Your dataset already supports this well.

Goal

Work with time hierarchies.

Exercise

Use:

- OrderDate

Build:

1. Sales by Year
2. Sales by Month

3. Trend line (continuous date)

Insight to notice

Tableau automatically creates:

- YEAR(OrderDate)
- MONTH(OrderDate)

This is where people usually go “oh... okay now I get it.”

4. Module 4 — Using Prebuilt Views

Now switch from DW tables to:

- `dbo.vw_SalesAnalysis`

This view already joins:

- Customers
- Employees
- Products
- Categories
- Suppliers

Goal

Understand **denormalized vs star schema usage**

Exercise

Build:

- Sales by CategoryName
- Sales by SupplierName
- Sales by EmployeeName

Reflection

You'll feel:

- Easier to use
- But less flexible for large-scale modeling

That trade-off is real in industry.

5. Module 5 — Dashboard Building

Goal

Combine multiple visuals.

Create:

1. Sales Trend (line chart)

2. Top Products (bar chart)
3. Sales by Country (map or bar)
4. KPI Cards:
 - Total Sales
 - Total Orders

Then assemble into a dashboard.

Add:

- Filters (Customer, Category, Date)
-

6. Module 6 — Aggregations & Performance

Now use:

- `DW.AggSalesMonthly`

Goal

Understand performance optimization

Exercise

Compare:

- Query using `FactSales`
vs
- Query using `AggSalesMonthly`

You'll notice:

- Faster performance
- Less granularity

That's the exact trade-off BI engineers design for.

7. Module 7 — Calculated Fields

Now it gets interesting.

Create:

- Profit proxy (since no cost):

```
SalesAmount * 0.3
```

- Average Order Value:

```
SUM(SalesAmount) / COUNTD(OrderID)
```

- Sales Category:

```
IF SUM(SalesAmount) > 1000 THEN "High"
ELSE "Low"
END
```

Goal

Start thinking, not just visualizing.

8. Module 8 — End-to-End Story

Now treat this like a real business case.

Scenario:

"Analyze sales performance across customers, products, and time."

Deliver:

A dashboard that answers:

- Who are top customers?
 - Which products drive revenue?
 - How does sales trend over time?
 - Which employee performs best?
-

9. (Optional but powerful) — ETL Awareness

You actually have:

- `sp_Load_DW`
- staging tables
- incremental logic

Why this matters

In real life:

- Tableau does NOT own data freshness
- ETL does

Exercise

- Run ETL
- Refresh Tableau extract
- Observe changes

That connection between pipeline and dashboard... that's where people usually level up.

Suggested Learning Flow (Simple)

Day 1

→ Modules 1–2

Day 2

→ Modules 3–4

Day 3

→ Modules 5–6

Day 4

→ Modules 7–8

Day 5

→ ETL + full project

Final Thought

Honestly... your dataset is already structured like a proper BI system:

- OLTP → STG → DW → Aggregate → Visualization

If you learn Tableau on top of *this*, you're not just learning a tool.

You're learning how reporting actually works in production.

I'll walk you through **3 concrete Tableau dashboards**, step by step, using your schema. These are the kind of dashboards you'd realistically show in a portfolio or in a real BI environment.

Dashboard 1 — Sales Overview (the “executive view”)

This is your main dashboard. Clean. High-level. Answers: *“How are we doing?”*

Step 1 — Connect the right data

Use:

- DW.FactSales
- Join:
 - DW.DimCustomer
 - DW.DimProduct
 - DW.DimEmployee

Relationships:

- FactSales.CustomerKey → DimCustomer.CustomerKey
 - FactSales.ProductKey → DimProduct.ProductKey
 - FactSales.EmployeeKey → DimEmployee.EmployeeKey
-

Step 2 — Create KPIs

Go to a new sheet.

KPI 1: Total Sales

- Drag SalesAmount → Text
- Aggregation: SUM

KPI 2: Total Quantity

- Drag Quantity → Text

KPI 3: Total Orders

- Drag OrderID → Text
- Change to: COUNTD

Format each as big numbers.

Step 3 — Sales Trend

New sheet.

- Columns → OrderDate (continuous MONTH)
- Rows → SUM(SalesAmount)

Optional:

- Add quick table calculation → Running Total

You should see a line chart forming over time.

Step 4 — Sales by Category (important)

You need Category from Product table.

- Rows → CategoryID (or name if available)
- Columns → SUM(SalesAmount)
- Sort descending

Turn into horizontal bar chart.

Step 5 — Top Customers

- Rows → CustomerName
 - Columns → SUM(SalesAmount)
 - Sort descending
 - Filter → Top 10 (by SUM SalesAmount)
-

Step 6 — Build Dashboard

Now go to Dashboard tab.

Add:

- KPI tiles at top
- Sales trend in middle
- Bar charts below

Add filters:

- OrderDate
- CategoryID
- CustomerName

Make filters "Apply to all worksheets".

Dashboard 2 — Product Performance

This one answers: *"What products actually drive revenue?"*

Switch data source if you want easier life:

Use:

- `dbo.vw_SalesAnalysis`

(It already joins everything nicely.)

Step 1 — Sales by Product

- Rows → ProductName
 - Columns → SUM(SalesAmount)
 - Sort descending
-

Step 2 — Add Quantity context

- Drag `Quantity` → Tooltip

So when you hover, you see volume vs revenue.

Step 3 — Scatter Plot (this is key)

This is where Tableau starts feeling powerful.

- Columns → SUM(Quantity)
- Rows → SUM(SalesAmount)
- Marks → Circle
- Add ProductName to Detail

Now you see:

- High quantity, low sales
 - Low quantity, high sales
 - etc.
-

Step 4 — Add Category filter

- Drag CategoryName → Filters
- Show filter

Now you can slice product performance by category.

Step 5 — Optional calculated field

Create:

“Sales per Unit”

```
SUM(SalesAmount) / SUM(Quantity)
```

Add to tooltip or color.

Step 6 — Build Dashboard

Include:

- Product ranking (bar)
- Scatter plot
- Category filter

This becomes a decision tool.

Dashboard 3 — Customer & Geographic Insights

This one feels more “business-facing”.

Step 1 — Sales by Country

Using `vw_SalesAnalysis` :

- Drag Country → view
- Drag SalesAmount → Color

If Tableau recognizes geography:

→ It becomes a map automatically

If not:

→ Use bar chart instead

Step 2 — Sales by Customer City

- Rows → CustomerCity
 - Columns → SUM(SalesAmount)
-

Step 3 — Customer segmentation

Create calculated field:

```
IF SUM(SalesAmount) > 5000 THEN "High Value"
ELSEIF SUM(SalesAmount) > 1000 THEN "Medium"
ELSE "Low"
END
```

Then:

- Rows → Segment
- Columns → COUNTD(CustomerName)

Step 4 — Sales by Employee

- Rows → EmployeeName
- Columns → SUM(SalesAmount)

This shows performance distribution.

Step 5 — Build Dashboard

Combine:

- Map or country chart
- Customer segmentation
- Employee performance

Add filters:

- Date
 - Country
-

Small but important habits (people skip these)

- Always format currency properly
 - Rename fields to human-readable names
 - Use tooltips intentionally (don't leave default clutter)
 - Keep dashboards clean... not crowded
-

If you follow all 3 dashboards...

You will have:

1. Executive summary
2. Product analytics
3. Customer insights

That's basically a full BI suite.

1. Business Case Story (Consulting Style)

Context

A mid-sized distribution company is struggling with fragmented reporting. Sales data exists, but:

- No unified dashboard
- No visibility into product performance
- No clear understanding of customer value
- Decisions are reactive, not data-driven

The company implemented a small data warehouse (`WAREHOUSE_SAMPLE`) to consolidate:

- Orders and transactions
- Customers and products
- Employees and suppliers

But... no one's really using it effectively yet.

That's where your work comes in.

Objective

Design a Tableau reporting solution that answers:

- How is the business performing overall?
 - Which products and categories drive revenue?
 - Who are the most valuable customers?
 - Where are growth opportunities?
-

Approach

Three analytical layers were developed:

1. Executive Overview

High-level KPIs and trends

2. Product Analytics

Revenue drivers and product efficiency

3. Customer Insights

Segmentation and geographic performance

Data sources:

- `DW.FactSales` (core metrics)
- `DW.Dim*` tables (dimensions)
- `vw_SalesAnalysis` (denormalized analysis view)

Key Findings

1. Sales Concentration

Revenue is highly concentrated in a small number of products.

- Top 10 products contribute a disproportionate share of total sales
- Long tail products generate minimal revenue

Implication:

Focus inventory and marketing on high-performing products.

2. Customer Segmentation

Customers fall into clear tiers:

- High-value customers (small group, high revenue)
- Medium-value customers (moderate contribution)
- Low-value customers (large group, low revenue)

Implication:

Retention strategies should prioritize high-value customers.

3. Seasonal Trends

Sales show variation over time:

- Certain months consistently outperform others
- Trend visualization suggests cyclical demand

Implication:

Align inventory and staffing with seasonal demand.

4. Employee Contribution

Sales performance varies significantly across employees.

- Some employees consistently generate higher revenue
- Others underperform

Implication:

Opportunity for performance benchmarking and training.

5. Geographic Distribution

Revenue is uneven across locations:

- Certain cities/countries dominate sales
- Others remain underdeveloped

Implication:

Potential expansion or targeted marketing opportunities.

Recommendations

1. Focus on top-performing products
 2. Develop customer retention strategies for high-value segments
 3. Align operations with seasonal trends
 4. Use employee performance data for coaching and incentives
 5. Explore growth opportunities in underperforming regions
-

Outcome

The Tableau dashboards provide:

- A single source of truth
 - Faster decision-making
 - Clear visibility into business drivers
-

2. Portfolio Project (What You Actually Show)

Now... this is where you make it *look real*.

You're not just showing dashboards.

You're telling a story.

Project Title

Sales Performance Analytics Dashboard — End-to-End BI Solution

Project Description (Put this in your portfolio)

This project demonstrates the design of an end-to-end business intelligence solution using SQL Server and Tableau.

A structured data warehouse was used to transform transactional sales data into analytical insights.

Interactive dashboards were developed to support executive decision-making, product analysis, and customer segmentation.

Architecture Section (important)

Explain briefly:

- Source: OLTP tables (`Orders` , `Products` , etc.)
- ETL: Stored procedure (`sp_Load_DW`)
- Data Warehouse:
 - Fact table: `FactSales`
 - Dimensions: Customer, Product, Employee
- Aggregation: `AggSalesMonthly`
- Visualization: Tableau dashboards

Keep it simple. No need to overdo.

Dashboard 1 Screenshot — Sales Overview

What to show:

- KPI cards (Total Sales, Orders, Quantity)
- Line chart (Sales trend)
- Bar chart (Sales by category)

Caption example

This dashboard provides a high-level view of business performance, including total sales, order volume, and trends over time.

Dashboard 2 Screenshot — Product Performance

Show:

- Top products bar chart
- Scatter plot (Quantity vs Sales)

Caption

This dashboard identifies top-performing products and analyzes the relationship between sales volume and revenue.

Dashboard 3 Screenshot — Customer Insights

Show:

- Customer segmentation
- Sales by geography

- Employee performance

Caption

This dashboard segments customers by value and highlights geographic and employee-level performance patterns.

Insights Section (short but sharp)

Don't write too much. Just key points:

- Revenue is concentrated among a small number of products
 - High-value customers contribute significantly to total sales
 - Sales trends indicate seasonal variation
 - Performance varies across employees
-

Tools Used

- SQL Server (data modeling, ETL)
 - Tableau Desktop (visualization)
 - Star schema modeling (Fact + Dimensions)
-

(Optional but powerful) Add this line

The project simulates a real-world BI environment, including ETL pipelines, staging layers, and dimensional modeling.

That line alone signals maturity.

Final Thought

What you've built here is not just a Tableau project.

It quietly shows:

- Data modeling
- ETL understanding
- Analytical thinking
- Visualization skills

Most people only show charts.

You're showing the *system behind the charts*.

Module 1 — Connecting Tableau to SQL Server (Step-by-Step)

Step 0 — Before you even open Tableau

Just a quick sanity check... this part trips people up.

Make sure:

- SQL Server is running
- Database `WAREHOUSE_SAMPLE` exists
- You can connect via SSMS

If SSMS works, Tableau will usually work too.

Step 1 — Open Tableau Desktop

Launch Tableau Desktop.

You'll land on the **Start Page**.

Left side → **Connect**

You'll see a list like:

- To a File
 - To a Server
-

Step 2 — Choose Microsoft SQL Server

Under **To a Server**:

Click → **Microsoft SQL Server**

A connection window pops up.

Step 3 — Fill Connection Details

Now you need to enter:

Server

- If local:

localhost

or

.\SQLEXPRESS

or your instance name (example):

DESKTOP-XXXX\MSSQLSERVER

Authentication

Choose one:

- **Windows Authentication** (easiest if local)
- or SQL Server Authentication (username/password)

Database (optional but recommended)

Type:

WAREHOUSE_SAMPLE

This avoids loading everything.

Click **Sign In**

Step 4 — You're Now in the Data Source Page

This is where things actually start to matter.

You'll see:

- Left panel → list of schemas (`dbo` , `DW` , `STG` , etc.)
- Middle → tables
- Top → canvas (where you drag tables)

Step 5 — Drag the Fact Table (Start Here)

From schema **DW**:

Drag:

FactSales

onto the canvas.

This becomes your base table.

Step 6 — Add Dimension Tables (Build Star Schema)

Now add:

1. DimCustomer

Drag `DimCustomer` next to `FactSales`

Tableau will try to create a relationship automatically.

Check it.

It should be:

```
FactSales.CustomerKey = DimCustomer.CustomerKey
```

2. DimProduct

Drag `DimProduct`

Relationship:

```
FactSales.ProductKey = DimProduct.ProductKey
```

3. DimEmployee

Drag `DimEmployee`

Relationship:

```
FactSales.EmployeeKey = DimEmployee.EmployeeKey
```

Step 7 — Verify Relationship Model (Important)

Look at the canvas:

You should see something like:

```
DimCustomer  DimProduct  DimEmployee
   \          |          /
    FactSales
```

This is your **star schema**.

If something looks off:

- Click the line between tables
 - Edit Relationship
 - Fix the key fields
-

Step 8 — Understand What Tableau Just Did

This part is subtle... but important.

Tableau is NOT doing physical joins (by default).
It's using **relationships (logical layer)**.

That means:

- Queries are generated dynamically
- Aggregations stay clean
- Less duplication issues

This is why star schema works beautifully here.

Step 9 — Go to Worksheet

Click:

Sheet 1

Now you're in the analysis view.

Step 10 — Sanity Check Your Data

Do a quick test:

- Drag **SalesAmount** → Rows
- Drag **OrderDate** → Columns

You should see a chart.

Then:

- Drag **CustomerName** → Rows

If it works without weird duplication...
your model is correct.

Common Mistakes (worth slowing down for a second)

1. Using raw tables like `Orders`

→ leads to messy joins and duplication

2. Using INNER JOIN manually

→ can break aggregation

3. Wrong keys

→ you'll see inflated numbers

4. Mixing staging tables

→ avoid `STG` schema entirely in Tableau

What You Just Achieved

You didn't just "connect Tableau".

You:

- Built a proper star schema
- Used fact + dimensions correctly
- Avoided duplication issues
- Set up a scalable model

This is already how BI teams structure data.

Quick Mental Model (keep this in your head)

- FactSales → numbers (what happened)
- Dimensions → context (who, what, where)

Tableau = combine both dynamically

this is the point where Tableau either *clicks...* or it feels random.

So instead of memorizing “dimension vs measure”, we’ll *see it happen*.

Open your Tableau workbook from Module 1. Stay connected to your star schema.

Module 2 — Understanding Measures vs Dimensions (Step-by-Step)

Step 1 — Look at the Data Pane (left side)

Pause here for a second.

On the left, Tableau splits fields into:

- **Dimensions** (top section)
- **Measures** (bottom section)

You should see things like:

Dimensions

- CustomerName
- City
- Country
- ProductName
- CategoryID
- EmployeeName
- OrderDate

Measures

- SalesAmount
 - Quantity
 - Price
-

Step 2 — What’s the real difference?

Let’s not overcomplicate it.

- **Dimensions** = how you slice data
- **Measures** = what you calculate

Or even simpler:

- Dimensions → labels
 - Measures → numbers
-

Step 3 — First Exercise: Total Sales

Go to a blank sheet.

Drag:

SalesAmount → Text (on Marks card)

Tableau will automatically do:

SUM(SalesAmount)

You'll see one big number.

That's your **Total Sales**.

Step 4 — Total Quantity

Now drag:

Quantity → Text

Tableau replaces the previous field.

Again, it shows:

SUM(Quantity)

So now you know:

→ Measures are automatically aggregated.

Step 5 — Combine Measures

Now drag:

- SalesAmount → Text
- Then drag Quantity → Text

Tableau will stack them.

You now have:

- Total Sales
- Total Quantity

This is basically your first KPI view.

Step 6 — Sales by Product (this is where it clicks)

Now:

- Drag `ProductName` → Rows
- Drag `SalesAmount` → Columns

Something interesting happens.

Instead of one number... you get many bars.

Why?

Because Tableau is now doing:

SUM(SalesAmount) per ProductName

You didn't write that.

It just *happened*.

Step 7 — Observe Carefully

This is the key idea:

When you add a **dimension**, Tableau:
→ splits the measure

So:

- No dimension → 1 total
- Add ProductName → split by product

That's the whole concept.

Step 8 — Sales by Customer

Replace ProductName:

- Drag `CustomerName` → Rows

Now you see:

SUM(SalesAmount) per Customer

Same measure. Different dimension.

Different story.

Step 9 — Add More Dimensions

Now try:

- Drag `Country` → Rows (above CustomerName)

Now you get:

- Sales by Country
 - then by Customer inside each country

You're layering context.

Step 10 — Play with OrderDate (important)

Drag:

OrderDate → Columns

Tableau might default to YEAR.

Now you get:

SUM(SalesAmount) per Year

Click the dropdown on OrderDate → try:

- Month
- Day
- Exact Date

Watch how the chart changes.

Step 11 — Discrete vs Continuous (quick intuition)

This part confuses people... so keep it simple.

- **Discrete (blue)** → categories
- **Continuous (green)** → timeline / axis

Try this:

- Right-click OrderDate
- Switch between:
 - Discrete Month
 - Continuous Month

You'll see the difference instantly.

Step 12 — Change Aggregation (important habit)

Click SUM(SalesAmount) in the view.

Change to:

- AVG
- MIN

- MAX

Watch the numbers change.

This tells you:

- Measures are flexible
 - Aggregation is not fixed
-

Step 13 — Common Mistake (watch for this)

Sometimes Tableau puts a numeric field into Dimensions.

Example: CategoryID

If that happens:

- Right-click → Convert to Dimension or Measure

Rule of thumb:

- IDs → Dimension
 - Amounts → Measure
-

Step 14 — What you should feel now

At this point... you should notice:

- You're not "building charts"
- You're *asking questions*

Like:

- Sales by what?
- Over what time?
- For which group?

And Tableau just responds.

Mini Exercise (do this without looking back)

Try to build:

1. Total Sales
2. Sales by Product
3. Sales by Customer
4. Sales by Country and Customer

If you can do that without hesitation...
you've passed this module.

Final Thought

This whole module boils down to one idea:

Measures give you the number.

Dimensions decide how that number is broken down.

Everything in Tableau builds on that.

this is where Tableau finally starts to feel like *real analysis*.

Up until now, you were slicing data.
Now you're watching it *move over time*.

Take it slow here... because once this clicks, dashboards become much easier.

Module 3 — Time-Based Analysis (Step-by-Step)

Step 1 — Start with a clean sheet

Open a new worksheet.

Don't drag anything yet. Just reset your brain a bit.

Step 2 — Drag OrderDate (watch carefully)

Drag:

OrderDate → Columns

Something automatic happens.

Tableau doesn't just show dates... it converts it into:

YEAR(OrderDate)

You'll see a pill like:

- YEAR(OrderDate) (usually blue)

This is Tableau building a **time hierarchy** for you.

Step 3 — Add SalesAmount

Now drag:

SalesAmount → Rows

You'll get a chart.

What you're seeing is:

SUM(SalesAmount) per Year

You didn't write that... but that's exactly what it is.

Step 4 — This is the “aha” moment

Pause here.

Tableau is doing two things automatically:

- Extracting YEAR from a date
- Aggregating Sales per year

This is why people go...

“oh... okay now I get it.”

Step 5 — Drill down to Month

Now click the small + icon on `YEAR(OrderDate)`.

This expands to:

YEAR → QUARTER → MONTH

Click until you reach **MONTH**.

Now your chart changes.

You’re seeing:

SUM(SalesAmount) per Month

Step 6 — Clean monthly view (important)

Sometimes Tableau shows months across multiple years in a weird way.

So instead, try this:

- Remove OrderDate
- Drag it again → Columns
- From dropdown, select:

MONTH(OrderDate) (continuous - green)

Now you get a proper timeline.

Step 7 — Build a proper trend line

Now your setup should be:

- Columns → continuous `MONTH(OrderDate)` (green)

- Rows → SUM(SalesAmount)

This gives you a **line chart over time**.

If it shows bars:

- Go to Marks → select **Line**
-

Step 8 — Add more context (optional but powerful)

Now drag:

- `CategoryID` → Color

or

- `ProductName` → Color

Now your line splits into multiple lines.

So now you're seeing:

- Sales trend per category
or
 - Sales trend per product
-

Step 9 — Continuous vs Discrete (this matters a lot)

Let's make this intuitive.

Discrete (blue)

- Separate buckets
- Example: Jan, Feb, Mar

Continuous (green)

- A flowing timeline
 - Like a real graph axis
-

Try this:

Click `MONTH(OrderDate)` →

Switch between:

- **Discrete Month (blue)**
- **Continuous Month (green)**

Watch what happens:

- Blue → separate bars
- Green → smooth timeline

This difference is *huge* in real dashboards.

Step 10 — Add Trend Line (actual analytics)

Now:

- Go to **Analytics tab** (next to Data)
- Drag **Trend Line** → into the chart

Tableau adds a trend.

Now you're not just visualizing...
you're *interpreting direction*.

Step 11 — Add filters (make it interactive)

Drag:

- `OrderDate` → Filters
- Choose range

Then:

- Right-click → Show Filter

Now you can zoom into time periods.

Step 12 — What you should notice

At this point, you should see:

- Time is not just another field
- It automatically creates hierarchy
- It changes how aggregation behaves

And most importantly:

Time turns static numbers into stories

Mini Exercise (do this yourself)

Try to build:

1. Sales by Year
2. Sales by Month
3. Line chart trend
4. Trend split by Category

If you can do all four...
you're already building real dashboards.

Common mistakes (quick reality check)

- Using discrete date for trends → looks broken
 - Mixing years unintentionally → confusing charts
 - Forgetting continuous axis → no real timeline
-

Final Thought

Without time, data is just totals.

With time...

you start seeing:

- growth
- decline
- patterns
- seasonality

That's where analysis actually begins.

Up to now, you were working with a **clean star schema**.

Now you're switching to a **prebuilt view**... which feels easier, but comes with trade-offs.

And then we'll go into something people rarely talk about...
why time charts can quietly mislead you.

4A — Using Prebuilt Views (`vw_SalesAnalysis`)

Step 1 — Switch Data Source

Go to your **Data Source** tab.

Instead of your DW tables:

- Remove existing tables (optional, or create new data source)

From schema `dbo` , drag:

`vw_SalesAnalysis`

That's it.

No joins. No relationships.

Everything is already flattened.

Step 2 — Look at the Data (pause here)

Scan the fields.

You'll see everything in one place:

- CustomerName
- CustomerCity, CustomerCountry
- EmployeeName
- ProductName
- CategoryName
- SupplierName
- Quantity
- Price
- SalesAmount

This is called a **denormalized dataset**.

Basically... someone already did the joins for you.

Step 3 — Sales by CategoryName

Go to a new sheet.

- Rows → `CategoryName`
- Columns → `SalesAmount`

Done.

You immediately get:

SUM(SalesAmount) per CategoryName

No thinking about joins.

Step 4 — Sales by SupplierName

Duplicate the sheet.

Replace:

- `CategoryName` → `SupplierName`

Now you see:

- Sales contribution by supplier
-

Step 5 — Sales by EmployeeName

Again:

- Rows → `EmployeeName`
- Columns → `SalesAmount`

Now you're analyzing performance.

Step 6 — Add sorting (quick polish)

For each chart:

- Click sort descending

This instantly makes insights clearer.

Step 7 — Add filter (optional but useful)

Try:

- Drag `CategoryName` → Filters
- Show Filter

Now your charts become interactive.

Step 8 — Notice how easy this feels

No joins

No relationships

No schema thinking

Just drag... and it works.

Step 9 — But... here's the catch

This is the part people miss.

Problem 1 — Less flexibility

You can't easily:

- Add new dimensions without changing the view
 - Control grain of data
-

Problem 2 — Hidden duplication risk

Because everything is flattened:

- Some aggregations can be misleading
 - You don't see the underlying structure
-

Problem 3 — Performance (at scale)

In real systems:

- Large views = slower queries
 - Star schema = more optimized
-

Simple Mental Comparison

Star Schema (DW tables)

- More setup
- More control
- Scales better

View (`vw_SalesAnalysis`)

- Easier
 - Faster to start
 - Less flexible
-

What you should feel

Honestly... most people prefer the view at first.

It feels natural.

But over time, you start missing control.

That tension is very real in industry.

4B — Why Some Time Charts Lie

This one... is subtle. And honestly a bit uncomfortable.

Because the chart looks correct.

But the story isn't.

Problem 1 — Mixing Years Without Realizing

Try this:

- Columns → `MONTH(OrderDate)` (discrete)
- Rows → `SUM(SalesAmount)`

You'll see:

- Jan, Feb, Mar...

Looks fine, right?

But...

You just combined:

- Jan 2023
- Jan 2024
- Jan 2025

Into ONE bar.

That's misleading.

Fix

Use:

- `YEAR(OrderDate) + MONTH(OrderDate)`
or
 - Continuous date (timeline)
-

Problem 2 — Truncated Time Range

If your data only has partial months:

Example:

- March only has 5 days loaded

But chart shows:

- March looks “low”

It’s not low... it’s incomplete.

Fix

- Always check data completeness
 - Use filters or annotations
-

Problem 3 — Wrong Aggregation Level

Example:

- Daily data aggregated into monthly
- But some days missing

This creates uneven comparisons.

Problem 4 — Using Discrete Instead of Continuous

Discrete month:

- Shows categories
- No real timeline

Continuous month:

- Shows actual progression

If you use the wrong one:

→ trend looks broken or artificial

Problem 5 — Ignoring Seasonality Context

A spike might look like growth...

But it might just be:

- Holiday season
- End-of-quarter push

Without context, charts mislead.

Problem 6 — Overplotting (too many lines)

If you add:

- ProductName → Color

You might get 50 lines.

Now:

- No pattern visible
 - Just noise
-

Fix

- Filter top N
 - Or aggregate
-

The uncomfortable truth

A chart doesn't lie.

But it can *mislead* very easily.

Because:

- It hides assumptions
 - It simplifies reality
 - It depends on how you build it
-

Practical habit (this is gold)

Every time you build a time chart, ask:

- Am I mixing years?
- Is the data complete?
- Is the aggregation correct?
- Is this discrete or continuous?

If you ask those 4 questions...

you'll already be ahead of most analysts.

Final Thought

Module 4 is really about contrast:

- Star schema → control

- View → convenience

And then...

Time analysis teaches you:

Even correct data can tell the wrong story if you're not careful.

You've built pieces already. Now you're turning them into something that *looks like a real dashboard someone would use*.

Not just charts thrown on a page... but something structured, intentional.

Let's build it step by step.

Module 5 — Real Dashboard Layout (Step-by-Step)

Step 0 — Before building the dashboard

You need 5 worksheets ready:

1. Total Sales (KPI)
2. Total Orders (KPI)
3. Sales Trend (line chart)
4. Top Products (bar chart)
5. Sales by Country (map or bar)

If not built yet, we'll build them quickly.

PART A — Build Each Component

1. KPI — Total Sales

New sheet → rename: `KPI - Total Sales`

- Drag `SalesAmount` → Text
- Make sure it shows:

```
SUM(SalesAmount)
```

Format:

- Increase font size (big... like 24–36)
- Format as currency

Optional:

- Edit title → "Total Sales"
-

2. KPI — Total Orders

New sheet → rename: `KPI - Total Orders`

- Drag `OrderID` → Text
- Change aggregation → COUNTD

Now it shows number of unique orders.

Format:

- Large font
 - Clean label
-

3. Sales Trend (Line Chart)

New sheet → rename: Sales Trend

- Columns → OrderDate (continuous MONTH)
- Rows → SalesAmount

Marks:

- Change to Line

Optional:

- Add trend line (Analytics tab)
-

4. Top Products (Bar Chart)

New sheet → rename: Top Products

- Rows → ProductName
- Columns → SalesAmount
- Sort descending

Filter:

- Top 10 products

Optional:

- Show labels
-

5. Sales by Country

New sheet → rename: Sales by Country

If Tableau detects geography:

- Drag Country → view
- Drag SalesAmount → Color

If not:

- Rows → Country
 - Columns → SalesAmount
-

PART B — Build the Dashboard

Step 1 — Create Dashboard

Click:

New Dashboard

Set size:

- Fixed size (e.g., 1200 x 800)
Keeps layout consistent.

Step 2 — Think Layout First (important)

Don't drag things randomly.

Use this structure:

[KPI Row]
[Trend Chart]
[Bottom Charts]

Simple. Clean. Works everywhere.

Step 3 — Add KPI Row

Drag a **Horizontal Container** to the top.

Inside it, place:

- KPI - Total Sales
- KPI - Total Orders

Resize them so they sit side-by-side.

Design tips (small but important)

- Remove sheet titles (or rewrite cleanly)
- Center align numbers
- Keep spacing balanced

Step 4 — Add Sales Trend

Drag **Sales Trend** below KPIs.

Make it wide.

This is your **main story chart**.

Step 5 — Add Bottom Charts

Drag a **Horizontal Container** under the trend.

Place:

- Top Products (left)
- Sales by Country (right)

Resize:

- Keep them balanced
 - Not too cramped
-

Step 6 — Add Filters

Now make it interactive.

From any worksheet:

- Drag `CustomerName` → Filters
- Drag `CategoryName` → Filters
- Drag `OrderDate` → Filters

Then:

- Right-click each → Show Filter
-

Step 7 — Apply Filters to All Charts

Important step.

For each filter:

- Click dropdown
- Apply to Worksheets → All Using This Data Source

Now everything syncs.

PART C — Polish (this is what separates beginners)

1. Clean Titles

Rename sheets to human language:

- "Sales Trend Over Time"
 - "Top 10 Products by Revenue"
 - "Sales by Country"
-

2. Consistent Formatting

- Currency → same format everywhere
 - Fonts → consistent
 - Colors → don't overdo
-

3. Reduce Noise

Remove:

- Gridlines (if distracting)
 - Unnecessary headers
 - Extra legends
-

4. Use White Space

Don't fill everything.

Let charts breathe.

This alone makes dashboards feel "professional".

5. Tooltips (quiet upgrade)

Hover over a chart → edit tooltip.

Example:

- Show SalesAmount
- Show Quantity

Keep it clean.

Final Layout (what you should see)

Something like:



[Total Sales] [Total Orders]

[Sales Trend Over Time]

[Top Products] [Sales by Country]

Simple. Clear. Effective.

What You Just Built

This is not just a dashboard.

It answers real questions:

- How are we performing overall?
 - Are we growing or declining?
 - What products drive revenue?
 - Where are our customers?
-

Common Mistakes (worth avoiding)

- Too many charts → clutter
 - Too many colors → confusion
 - Filters not synced → inconsistent views
 - Tiny KPIs → no impact
-

Final Thought

A good dashboard isn't about showing everything.

It's about showing *just enough*... clearly.

If someone can look at it for 10 seconds and understand what's going on...
you've done it right.

this module is a bit different.

Less about visuals... more about *how data behaves underneath*.

This is where you start thinking like a BI engineer, not just a dashboard builder.

Let's go step by step and actually *feel* the difference.

Module 6 — Aggregations & Performance (Step-by-Step)

Step 0 — What you're comparing

You have two options:

1. Detailed table

DW.FactSales

- Row-level data
- Very flexible
- Slower at scale

2. Aggregated table

DW.AggSalesMonthly

- Pre-summarized
 - Fast
 - Limited detail
-

PART A — Using FactSales (Detailed Data)

Step 1 — Create new worksheet

Rename:

Sales Trend - Fact

Step 2 — Build monthly trend manually

Drag:

- OrderDate → Columns
- Change to → MONTH (continuous)

Then:

- SalesAmount → Rows

Now you have:

SUM(SalesAmount) per Month

Step 3 — Add Year (important)

Because FactSales has full dates:

- Drag OrderDate again → Columns (before Month)
- Set to YEAR

Now you're seeing:

- Year → Month breakdown
-

Step 4 — Notice flexibility

You can now:

- Drill down to day
- Add Customer
- Add Product
- Slice any way you want

This is *full granularity*.

Step 5 — Observe performance

If your dataset is small, it feels fast.

But imagine:

- Millions of rows
- Complex joins

This would slow down.

PART B — Using AggSalesMonthly (Pre-Aggregated)

Step 1 — Switch data source

Add new data source → select:

DW.AggSalesMonthly

Step 2 — Build same chart

New worksheet → rename:

Sales Trend - Agg

Drag:

- SalesYear → Columns
- SalesMonth → Columns (next to year)
- TotalSales → Rows

Step 3 — Clean it up

Right now it may look messy.

Fix it:

- Convert Year + Month into a proper timeline feel
- Or keep as discrete hierarchy

You'll get:

SUM(TotalSales) per Year/Month

Step 4 — Compare visually

Now compare both sheets:

- Sales Trend - Fact
- Sales Trend - Agg

They should look similar.

PART C — Feel the Difference

1. Performance

Even if you don't notice it strongly now...

Conceptually:

- FactSales → scans many rows
- AggSalesMonthly → scans very few rows

So:

→ Aggregated table is faster

2. Granularity (this is the real trade-off)

Try this:

On FactSales:

- Add `ProductName`

Works fine.

On AggSalesMonthly:

Try adding `ProductName`...

You can't.

Because:

→ That detail is gone

3. Flexibility vs Speed

This is the core idea:

Feature	FactSales	AggSalesMonthly
Detail level	High	Low
Performance	Slower	Faster
Flexibility	High	Limited

PART D — Real BI Thinking

This is not just a Tableau thing.

This is how real systems are designed.

Why aggregated tables exist

Because:

- Executives don't need row-level detail
- Dashboards must load fast
- Data volumes grow huge

So engineers create:

→ Pre-aggregated tables (like yours)

When to use each

Use FactSales when:

- You need drill-down
 - You explore data
 - You build flexible dashboards
-

Use AggSalesMonthly when:

- You need fast loading
 - You show high-level trends
 - You present to executives
-

PART E — Small Experiment (do this)

Try building:

From FactSales:

- Sales by Month
 - Then add Product filter
-

From AggSalesMonthly:

- Try same thing

You'll immediately feel:

→ "I can't go deeper here"

That's the limitation.

Common Mistakes

- Using Fact table for everything → slow dashboards
 - Using aggregate table for detailed analysis → missing insights
 - Mixing both incorrectly → inconsistent numbers
-

Final Thought

This module is really about one idea:

You can have speed... or detail... but rarely both at the same time.

Good BI design is about choosing the right balance.

this is where Tableau stops being just drag-and-drop...
and starts feeling like *thinking with data*.

Calculated fields are basically:
"what if the data doesn't already give me what I need?"

Let's build them step by step.

Module 7 — Calculated Fields (Step-by-Step)

Step 1 — Where to create calculations

In Tableau:

- Go to **Data Pane** (left side)
- Right-click anywhere
- Click:

Create → Calculated Field

A window opens.

You give:

- Name
 - Formula
-

PART A — Core Calculations

1. Profit Proxy (simple but important)

Step

Create new calculated field:

Name:

Profit Proxy

Formula:

[SalesAmount] * 0.3

Click OK.

Use it

- Drag Profit Proxy → Rows
- Or use it in KPI

Now you've created a *new metric* that didn't exist.

2. Average Order Value (AOV)

Step

Create:

Name:

Avg Order Value

Formula:

`SUM([SalesAmount]) / COUNTD([OrderID])`

Use it

- Drag to Text → shows one number
- Or break it by Customer

Now you're answering:

→ "How much does each order generate on average?"

3. Sales Category (segmentation)

Step

Create:

Name:

Sales Category

Formula:

```
IF SUM([SalesAmount]) > 1000 THEN "High"
ELSE "Low"
END
```

Use it

- Drag `Sales Category` → Rows
- Drag `SalesAmount` → Columns

You'll see High vs Low groups.

PART B — Important Insight (pause here)

You just did 3 different types of thinking:

- Arithmetic → Profit Proxy
- Aggregation → Avg Order Value
- Logic → Sales Category

That's basically 80% of real-world analytics.

PART C — More Useful Calculations (this is where it gets real)

Let's add more... the kind people actually use.

4. Sales per Customer

Name:

Sales per Customer

Formula:

`SUM([SalesAmount]) / COUNTD([CustomerID])`

Use:

- KPI or comparison
-

5. Quantity per Order

Name:

Avg Quantity per Order

Formula:

`SUM([Quantity]) / COUNTD([OrderID])`

6. Price Banding (very practical)

Name:

Price Category

Formula:

```
IF [Price] < 50 THEN "Low Price"  
ELSEIF [Price] < 200 THEN "Medium Price"  
ELSE "High Price"  
END
```

Use:

- Analyze product segmentation

7. Top N Flag (super useful)

Name:

Top Product Flag

Formula:

```
IF RANK(SUM([SalesAmount])) <= 10 THEN "Top 10"  
ELSE "Others"  
END
```

Use:

- Filter or color

8. Running Total (time-based intelligence)

Instead of formula, use:

- Right-click `SalesAmount`
- Quick Table Calculation → Running Total

But you can also define logic around it.

9. Growth Rate (this is powerful)

Name:

Sales Growth %

Formula:

```
(SUM([SalesAmount]) - LOOKUP(SUM([SalesAmount]), -1))  
/ LOOKUP(SUM([SalesAmount]), -1)
```

This compares current vs previous period.

10. Customer Lifetime Value (simple version)

Name:

Customer Lifetime Value

Formula:

```
SUM([SalesAmount])
```

Then:

- Use with CustomerName

(Yes... simple, but effective)

PART D — Use Calculations in Visuals

Now combine them.

Example 1 — Profit by Product

- Rows → ProductName
 - Columns → Profit Proxy
-

Example 2 — AOV by Customer

- Rows → CustomerName
 - Columns → Avg Order Value
-

Example 3 — Sales Category Distribution

- Rows → Sales Category
 - Columns → COUNTD(CustomerName)
-

PART E — Common Mistakes (important)

1. Mixing aggregated and non-aggregated

Wrong:

```
[SalesAmount] / COUNTD([OrderID])
```

Correct:

```
SUM([SalesAmount]) / COUNTD([OrderID])
```

2. Using SUM inside row-level logic incorrectly

Example:

```
IF SUM([SalesAmount]) > 1000
```

Works... but behaves differently depending on context.

3. Forgetting context

A calculation changes meaning depending on:

- What dimensions are in the view
- How data is grouped

Same formula... different result.

PART F — What just happened (important mindset shift)

Before:

→ You were using existing data

Now:

→ You are *creating new meaning from data*

That's a big shift.

Final Thought

Calculated fields are where analysis actually begins.

Because real business questions are rarely:

- "What is total sales?"

They are:

- "Which customers are high value?"
- "Are we growing?"
- "Which products are efficient?"

And those answers don't exist... until you define them.

integrate these calculations into your final dashboard (making it smarter)... now we take your dashboard from “nice visuals” to something that actually *thinks*.

Right now your dashboard shows:

- totals
- trends
- breakdowns

After this... it will:

- explain performance
- highlight patterns
- guide decisions

Let’s upgrade it step by step.

Goal — Make Your Dashboard Smarter

We will integrate:

- Profit Proxy
- Average Order Value (AOV)
- Sales Category (segmentation)
- Optional: Growth / Top Product logic

Into your existing dashboard.

Step 0 — Make sure calculations exist

You should already have:

```
Profit Proxy = [SalesAmount] * 0.3
Avg Order Value = SUM([SalesAmount]) / COUNTD([OrderID])
Sales Category = IF SUM([SalesAmount]) > 1000 THEN "High" ELSE "Low" END
```

If not, create them first.

PART A — Upgrade KPI Section

Step 1 — Add Profit KPI

Duplicate your “Total Sales” sheet.

Rename:

KPI - Profit

Replace:

- SalesAmount → Profit Proxy

Format:

- Currency
 - Slightly smaller than Total Sales
-

Step 2 — Add AOV KPI

New sheet:

- Drag Avg Order Value → Text

Rename:

KPI - Avg Order Value

Step 3 — Update Dashboard Layout

Go to dashboard.

Top row becomes:

[Total Sales] [Total Orders] [Profit] [AOV]

Resize so they align evenly.

Why this matters

Now your dashboard answers:

- Revenue (Sales)
- Volume (Orders)
- Efficiency (AOV)
- Value (Profit proxy)

That's already more "business-like".

PART B — Upgrade Sales Trend

Step 4 — Add Profit Trend

Go to your Sales Trend sheet.

Drag:

- Profit Proxy → Rows

Now you have two options:

Option A — Dual Axis (recommended)

- Right-click second axis → Dual Axis
- Synchronize axis

Now you see:

- Sales trend
 - Profit trend
-

Step 5 — Add Growth Insight (optional but powerful)

Use your **Sales Growth %** calculation.

- Drag it to Tooltip

Now when hovering:

→ You see growth rate

PART C — Upgrade Product Analysis

Step 6 — Add Profit to Top Products

Go to **Top Products** sheet.

- Drag **Profit Proxy** → Tooltip

Optional:

- Drag to Color

Now you see:

- High sales but low profit
- Or balanced products

That's a *real* business insight.

Step 7 — Add Price Category

- Drag **Price Category** → Color

Now products are grouped into:

- Low / Medium / High price

You've added segmentation.

PART D — Add Customer Intelligence

Step 8 — Create Customer Segmentation View

New sheet:

- Rows → Sales Category
- Columns → COUNTD(CustomerName)

Rename:

Customer Segmentation

Step 9 — Add to Dashboard

Place this chart:

- Either next to Top Products
- Or below it

Now your dashboard shows:

- Not just sales...
 - But *who contributes to it*
-

PART E — Make Filters Smarter

Step 10 — Add calculation-aware filters

You already have:

- Customer
- Category
- Date

Now add:

- Sales Category (High / Low)

This allows users to:

→ Focus only on high-value segments

PART F — Improve Tooltips (hidden power)

Step 11 — Customize tooltips

Go into:

- Sales Trend
- Top Products

Edit Tooltip.

Add:

- SalesAmount
- Profit Proxy

- Avg Order Value

Example idea:

```
Sales: <SUM(SalesAmount)>
Profit: <SUM(Profit Proxy)>
```

Now hovering gives *context*, not just numbers.

PART G — Highlight Logic (very effective)

Step 12 — Highlight Top Products

Use your Top Product Flag:

- Drag to Color

Now:

- Top 10 → highlighted
- Others → muted

This guides attention instantly.

PART H — Final Layout (Smart Dashboard)

Now your dashboard looks like:

```
[ Sales ] [ Orders ] [ Profit ] [ AOV ]
```

```
[ Sales + Profit Trend ]
```

```
[ Top Products (with Profit + Category) ] [ Sales by Country ]
```

```
[ Customer Segmentation ]
```

What Changed (this is the key insight)

Before:

- What happened?

Now:

- How valuable is it?
 - Who drives it?
 - Is it efficient?
 - Is it growing?
-

Common Mistakes When Adding Calculations

- Adding too many metrics → clutter
- Mixing definitions (e.g., profit logic unclear)
- Not explaining calculated fields

Keep it:

- Clear
 - Intentional
-

Final Thought

A “normal” dashboard shows data.

A “smart” dashboard answers questions before they’re asked.

That’s what you just built.

Module 8 — End-to-End Story (Step-by-Step)

Scenario

You are given a simple instruction:

“Analyze sales performance across customers, products, employees, and time.”

No one tells you what charts to build.

You decide that.

Step 1 — Define the Questions (before Tableau)

This is the part people skip... but it matters most.

You need to answer:

1. Who are top customers?
2. Which products drive revenue?
3. How does sales trend over time?
4. Which employee performs best?

That's your structure.

Everything you build should map to one of these.

Step 2 — Prepare Your Data Source

Use your **star schema**:

- DW.FactSales
- DW.DimCustomer
- DW.DimProduct
- DW.DimEmployee

Make sure relationships are correct.

Step 3 — Build Each Business View

We'll build 4 core views.

View 1 — Sales Trend Over Time

Build

- Columns → OrderDate (MONTH continuous)
- Rows → SalesAmount

Optional:

- Add Profit Proxy (dual axis)
-

What it answers

- Are we growing?
 - Any seasonal patterns?
-

View 2 — Top Customers

Build

- Rows → CustomerName
- Columns → SalesAmount
- Sort descending

Filter:

- Top 10
-

Upgrade (important)

Add:

- Avg Order Value → Tooltip
 - Sales Category → Color
-

What it answers

- Who brings the most revenue?
 - Are they high-value or low-value customers?
-

View 3 — Product Performance

Build

- Rows → ProductName
- Columns → SalesAmount

Sort:

- Descending
-

Upgrade

Add:

- Profit Proxy → Color
 - Price Category → Color or Filter
-

What it answers

- Which products drive revenue?
 - Are high-revenue products also high-profit?
-

View 4 — Employee Performance

Build

- Rows → EmployeeName
 - Columns → SalesAmount
-

Upgrade

Add:

- COUNTD(OrderID) → Tooltip
 - Optional: Avg Order Value
-

What it answers

- Who sells the most?
 - Who handles more orders?
-

Step 4 — Build Supporting KPIs

Create KPI sheets:

- Total Sales
 - Total Orders
 - Profit Proxy
 - Avg Order Value
-

Step 5 — Assemble the Dashboard

Now go to Dashboard view.

Layout Structure (very intentional)

[KPI Row]

[Sales Trend]

[Customers] [Products]

[Employee Performance]

Step 6 — Add Filters

Add:

- Customer
- Product Category
- OrderDate

Apply to all worksheets.

Step 7 — Add Context (this is what makes it a “story”)

Now... don't just show charts.

Guide the user.

Add Titles

Instead of:

- “Sheet 1”

Use:

- “Sales Trend Over Time”
- “Top Customers by Revenue”
- “Top Products Driving Sales”
- “Employee Sales Performance”

Add Short Descriptions (optional but powerful)

Example:

- “Top customers contribute majority of revenue”

- "Sales trend shows monthly fluctuations"
-

Step 8 — Read Your Dashboard Like a Story

Now pause.

Look at your dashboard from top to bottom.

What it should tell:

1. Overall Performance

- Total Sales
 - Total Orders
 - Profit
 - AOV
-

2. Trend

- Are we growing or declining?
-

3. Drivers

- Customers → who buys
 - Products → what sells
-

4. Execution

- Employees → who delivers results
-

Step 9 — Validate (very real step)

Ask yourself:

- Can someone understand this in 15 seconds?
- Can they answer the 4 business questions?
- Is anything confusing?

If yes → simplify.

Step 10 — Final Polish

- Align everything neatly
- Keep colors consistent
- Remove unnecessary clutter
- Use spacing

This is what separates:

- student work
vs
 - professional dashboard
-

Final Result (What You Built)

You didn't just build charts.

You built a **business narrative**:

- Revenue overview
 - Growth pattern
 - Customer value
 - Product contribution
 - Employee performance
-

Final Thought

At this stage, the skill isn't Tableau anymore.

It's:

Knowing what matters... and showing it clearly.

That's what companies actually look for.

this one feels a bit “behind the scenes”...
but honestly, this is where people start standing out.

Because now you’re not just building dashboards...
you’re understanding *where the data comes from and how it changes*.

Let’s walk through it like you’re actually operating a real system.

Module 9 — ETL Awareness (Step-by-Step)

First... what you already have

Your database is not just tables.

It already includes:

- Stored procedure → `sp_Load_DW`
- Staging tables → `STG.*`
- Data warehouse → `DW.*`
- Incremental logic → based on `LastModified`

That’s basically a mini real-world pipeline.

Step 1 — Understand the Flow (simple version)

Before touching Tableau, just *get the idea*:

```
Raw Tables (dbo)
  ↓
Staging (STG)
  ↓
Dimensions (DW.Dim*)
  ↓
Fact Table (DW.FactSales)
  ↓
Aggregates (DW.AggSalesMonthly)
  ↓
Tableau Dashboard
```

So:

Tableau is just the *last step*.

Step 2 — Run the ETL Process

Open **SQL Server Management Studio (SSMS)**.

Run:

```
EXEC dbo.sp_Load_DW;
```

(or `DW.sp_Load_DW`, depending on which version you use)

What just happened (quietly)

Behind the scenes:

- New/updated data copied into staging
- Dimensions updated (Customer, Product, Employee)
- FactSales refreshed
- Monthly aggregates rebuilt
- ETL log updated

You didn't see it... but a lot happened.

Step 3 — Validate Data Changed

Run a quick check:

```
SELECT TOP 10 * FROM DW.FactSales ORDER BY SalesKey DESC;
```

And:

```
SELECT * FROM DW.AggSalesMonthly;
```

You're confirming:

→ Data exists and is updated

Step 4 — Connect Tableau to This Pipeline

Now go back to Tableau Desktop.

Step 5 — Refresh Data Source

If you're using **Live connection**:

- Just click:

Data → Refresh

If you're using **Extract (.hyper)**:

- Click:

Data → Extract → Refresh

What happens now

Tableau re-queries:

- `DW.FactSales`
- or `DW.AggSalesMonthly`

And pulls updated data.

Step 6 — Observe Changes (this is the key exercise)

Now go to your dashboard.

Look for:

- KPI changes (Sales, Orders, Profit)
- Trend shifts
- New customers/products appearing

If nothing changes...

That's actually a lesson too.

It means:

→ No new data passed the ETL filter (`LastModified`)

Step 7 — Simulate Real Change (important)

Let's force a change.

Run in SQL:

```
UPDATE dbo.Orders  
SET LastModified = GETDATE()
```

```
WHERE OrderID = 1;
```

Then:

```
EXEC dbo.sp_Load_DW;
```

Now go back to Tableau → Refresh.

You should see:

→ small but real changes

Step 8 — Check ETL Log (this is very real-world)

Run:

```
SELECT * FROM dbo.ETL_Log ORDER BY StartTime DESC;
```

You'll see:

- StartTime
- EndTime
- RowsProcessed
- Status

This is what data teams monitor daily.

Step 9 — Understand Incremental Logic (big concept)

Your ETL does this:

```
WHERE LastModified > @LastETLTime
```

Meaning:

- Only new/changed data is processed
 - Not full reload
-

Why this matters

Without this:

- Every refresh = full table scan
- Very slow
- Not scalable

With this:

- Fast
 - Efficient
 - Production-ready
-

Step 10 — Connect It Back to Tableau

Now the key realization:

Tableau does NOT decide when data updates.

ETL does.

Real-world flow

```
ETL runs at 2 AM
↓
Warehouse updated
↓
Tableau refresh at 3 AM
↓
Dashboard shows new data
```

Step 11 — What You Just Learned (important)

You now understand:

- Data doesn't magically appear in dashboards
 - There's a pipeline behind it
 - Freshness depends on ETL timing
 - Aggregations may also depend on ETL
-

Common Mistakes (very real)

- Thinking Tableau = source of truth

- Ignoring data latency
 - Not validating ETL success
 - Refreshing dashboard but ETL hasn't run
-

Final Thought

This is the moment people usually level up.

Because now you see:

Dashboard = surface

ETL = engine

And if the engine isn't running...
the dashboard is just showing old truth.

1. Real Stakeholder Questions (Using Your Dashboard)

Imagine someone opens your dashboard and asks:

Question 1

"Why did sales drop last month?"

How you answer:

- Go to **Sales Trend**
- Filter to last 2–3 months
- Then:
 - Check Top Products → did a key product drop?
 - Check Customers → did a major customer disappear?

👉 You're not guessing. You're *navigating the dashboard*.

Question 2

"Who are our most valuable customers?"

Answer:

- Use **Top Customers view**
- Sort by SalesAmount
- Add:
 - Avg Order Value
 - Sales Category

👉 Now you say:

"Top customers are high revenue but also high AOV"

Question 3

"Are we growing consistently?"

Answer:

- Show Sales Trend
- Add Growth %

👉 You don't just say yes/no

You say:

"Growth is uneven... strong in Q2, flat in Q3"

2. Real Performance Problems (and Fixes)

Now... things break in real life.

Problem 1 — Dashboard is slow

Symptoms:

- Takes 10+ seconds to load
- Filters lag

Causes

- Using `FactSales` (too many rows)
 - Too many joins
 - Too many calculations
-

Fixes

Fix 1 — Use aggregated table

Switch to:

`DW.AggSalesMonthly`

Fix 2 — Reduce filters

Too many filters = too many queries

Fix 3 — Use extract

Instead of live connection:

- Create `.hyper` extract
-

Fix 4 — Simplify calculations

Avoid heavy `LOOKUP`, `RANK` everywhere

3. Simulate Real Business Questions (Build Calculations Live)

Now imagine someone asks:

Question

“Which customers are high value but low frequency?”

You don't have that.

So you build it.

Step 1 — Create Frequency

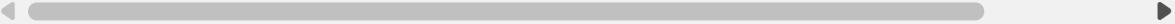
```
Order Frequency = COUNTD([OrderID])
```

Step 2 — Create Value

```
Customer Value = SUM([SalesAmount])
```

Step 3 — Create Segment

```
IF [Customer Value] > 1000 AND [Order Frequency] < 5 THEN "High Value - Low  
ELSE "Other"  
END
```



Now visualize:

- Rows → CustomerName
- Color → Segment

👉 You just answered a business question *on the spot*

4. Simulate Stakeholder Q&A (Live)

Let's do this like a meeting.

Stakeholder:

"Why are some high-selling products not profitable?"

You:

- Go to Top Products

- Add Profit Proxy to color

Then say:

"Some products have high revenue but lower profit margins... likely due to pricing structure"

Stakeholder:

"Which employee performs best?"

You:

- Open Employee chart
- Sort by SalesAmount

Then add:

- Avg Order Value

👉 Insight:

"One employee sells fewer orders but higher value ones"

That's how conversations actually go.

5. Simulate Production Issue (Wrong Numbers)

This is *very real*.

Scenario

Dashboard shows:

- Total Sales = 1,000,000

But SQL shows:

- 800,000
-

Step-by-step debugging

Step 1 — Check Tableau aggregation

Are you using:

SUM(SalesAmount)

or something weird?

Step 2 — Check joins

Common issue:

- Fact joined incorrectly → duplicates

👉 Look for row multiplication

Step 3 — Check filters

Maybe Tableau has:

- Date filter
- Customer filter

SQL doesn't

Step 4 — Check data source

Is Tableau using:

- FactSales
- or
- vw_SalesAnalysis

These may differ

Step 5 — Check ETL freshness

Run:

```
SELECT * FROM ETL_Control;
```

👉 Maybe Tableau is showing older data

Step 6 — Validate at row level

Pull small sample:

```
SELECT OrderID, SUM(SalesAmount)
FROM DW.FactSales
GROUP BY OrderID
```

Compare with Tableau.

👉 This is exactly how real debugging happens.

6. Full BI Architecture (Your Project)

Let's zoom out.

Your project already has a full pipeline.

Architecture Flow

```
graph TD; A["[ Source System (dbo tables) ]"] --> B["[ Staging Layer (STG) ]"]; B --> C["[ Data Warehouse (DW) ]"]; C --> D["[ Aggregation Layer ]"]; D --> E["[ ETL Process ]"]; E --> F["[ BI Layer ]"]; F --> G["Tableau, Dashboard"];
```

[Source System (dbo tables)]

↓

[Staging Layer (STG)]

↓

[Data Warehouse (DW)]

- DimCustomer
- DimProduct
- DimEmployee
- FactSales

↓

[Aggregation Layer]

- AggSalesMonthly

↓

[ETL Process]

- sp_Load_DW

↓

[BI Layer]

Tableau, Dashboard

What each layer does

Source (dbo)

Raw operational data

Staging (STG)

Temporary clean area

DW (star schema)

Optimized for analysis

Aggregates

Performance optimization

ETL

Controls:

- data movement
 - freshness
 - transformations
-

Tableau

Visualization layer only

Final Insight (this is the real takeaway)

At this point... you're not just:

- building charts
- writing SQL

You're thinking in systems:

- data pipeline
 - transformation
 - performance
 - business questions
-

And that's the difference.

Someone who knows Tableau can build dashboards.

Someone who understands all this...
can explain, debug, optimize, and *own the data*.